

LangChain Chain with Prompt Template and Conversation Memory

Objective

The objective of this task was to build a simple LangChain chain for a hospitality use case. The chain uses a **prompt template** and **conversation memory** to remember information provided by the guest.

The task has two steps:

1. Collect the guest's booking details.
2. Confirm the booking using the information from the conversation.

Hospitality Use Case

The chatbot acts as a **hotel booking assistant**. It remembers details such as the guest's name, check-in date, check-out date, number of guests, and room type.

Python Code

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
from langchain.memory import ConversationBufferMemory

# Create the LLM
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Create conversation memory
memory = ConversationBufferMemory(
    return_messages=True,
```

```

        memory_key="chat_history"
    )

# Create prompt template
prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        """"You are a helpful hotel booking assistant.
        Help the guest book a hotel room.
        Remember details provided earlier in the conversation."""
    ),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}")
])

# Create the chain
chain = prompt | llm

# Step 1: Collect booking details
user_input = "My name is Rahul. I need a Deluxe room from 15 to 17
September for 2 guests."

history = memory.load_memory_variables({})["chat_history"]

response = chain.invoke({
    "input": user_input,
    "chat_history": history
})

print("Assistant:", response.content)

memory.save_context(
    {"input": user_input},
    {"output": response.content}
)

# Step 2: Confirm the booking
user_input = "Please confirm my booking."

```

```
history = memory.load_memory_variables({})["chat_history"]

response = chain.invoke({
    "input": user_input,
    "chat_history": history
})

print("Assistant:", response.content)
```

How It Works

Step 1 – Collect Details

The guest provides information such as:

"My name is Rahul. I need a Deluxe room from 15 to 17 September for 2 guests."

The chatbot processes the information and stores the conversation in memory.

Step 2 – Confirm Booking

The guest then says:

"Please confirm my booking."

The chatbot uses the previous conversation from memory and understands that the guest wants a **Deluxe room for 2 guests from 15 to 17 September**.

This means the guest does not need to repeat all the information again.

Main Components

Prompt Template

The prompt template defines how the chatbot should behave and what information it should use while responding.

Conversation Memory

ConversationBufferMemory stores the previous conversation so the chatbot can remember information provided earlier.

LangChain Chain

The chain connects the prompt template with the language model:

```
chain = prompt | llm
```

It takes the user's message and conversation history and sends them to the AI model.

Expected Output

Step 1:

Assistant: Thanks Rahul. I have noted your request for a Deluxe room for 2 guests from 15 to 17 September. Would you like me to confirm the booking?

Step 2:

Assistant: Sure, Rahul. Your Deluxe room booking for 2 guests from 15 to 17 September is ready for confirmation.

Key Learning

Through this task, I learned how LangChain can be used to build a simple conversational application. I understood how **prompt templates provide instructions**, while **conversation memory helps the chatbot remember previous messages**. This can be useful in hospitality applications where guests may provide booking details across multiple messages.